МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДВНЗ «КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ ЕКОНОМІЧНИЙ УНІВЕРСИТЕТ ІМЕНІ ВАДИМА ГЕТЬМАНА»

Навчально-науковий інститут «Інститут інформаційних технологій в економіці»

Кафедра інформаційних систем в економіці

ЗВІТ
Лабораторна робота №2
з дисципліни «Системи і методи штучного інтелекту»

Виконав (-ла):
студент (-ка) 4 курсу, групи ІН-304
Коксюк Олег Віталійович

Київ – 2026

МЕТА ТА ЗАВДАННЯ РОБОТИ

Метою лабораторної роботи є дослідження та порівняння різних методів класифікації даних засобами мови програмування Python з використанням бібліотеки scikit-learn.

Для досягнення мети вирішувались такі завдання:

– побудувати бінарний класифікатор на основі лінійного метода опорних векторів (LinearSVC) для датасету Census Income;

– порівняти якість класифікації при використанні різних ядер SVM (поліноміального, гаусового, сигмоїдального);

– дослідити та порівняти шість алгоритмів класифікації на датасеті Iris за допомогою крос-валідації;

– порівняти ті самі алгоритми на датасеті Census Income;

– застосувати лінійний класифікатор Ridge до датасету Iris та розрахувати додаткові метрики якості.

ЗАВДАННЯ 2.1. КЛАСИФІКАЦІЯ ЗА ДОПОМОГОЮ SVM (LINEARSVC)

2.1.1. Опис датасету Census Income

Для вирішення задачі бінарної класифікації використовувався набір даних Census Income (Adult). Мета – прогнозування того, чи перевищує річний дохід людини 50 000 доларів США на основі 14 демографічних і соціально-економічних ознак.

Цільова змінна: income ∈ {«<=50K», «>50K»}.

Таблиця 2.1 – Опис ознак датасету Census Income

№	Ознака	Тип	Опис
1	**age**	числова	Вік (роки)
2	**workclass**	категоріальна	Тип зайнятості / роботодавця
3	**fnlwgt**	числова	Ваговий коефіцієнт перепису
4	**education**	категоріальна	Рівень освіти
5	**education-num**	числова	Освіта у вигляді числа (роки/ранг)
6	**marital-status**	категоріальна	Сімейний стан
7	**occupation**	категоріальна	Професія / вид робіт
8	**relationship**	категоріальна	Роль у сім'ї
9	**race**	категоріальна	Раса
10	**sex**	категоріальна	Стать
11	**capital-gain**	числова	Дохід від капіталу
12	**capital-loss**	числова	Втрати капіталу
13	**hours-per-week**	числова	Годин на тиждень

14	**native-country**	категоріальна	Країна походження

2.1.2. Попередня обробка даних

Оскільки набір даних містить обидва типи ознак – числові та категоріальні – перед навчанням моделі виконувалась попередня обробка:

– числові ознаки (age, fnlwgt, education-num, capital-gain, capital-loss, hours-per-week) – заповнення пропусків медіаною та масштабування StandardScaler;

– категоріальні ознаки (workclass, education, marital-status, occupation, relationship, race, sex, native-country) – заповнення найчастішим значенням та OneHotEncoder;

– рядки зі значеннями «?» видалялись (dropna);

– балансування вибірки: до 25 000 прикладів на кожен клас;

– розбиття 80/20 (train/test) зі стратифікацією.

Обробку реалізовано через ColumnTransformer та Pipeline, що забезпечує уніфіковану підготовку даних для всіх моделей.

```
import os


import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
```

```
    f1_score,
    precision_score,
    recall_score,
)
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import LinearSVC


def load_income_data(path: str) -> pd.DataFrame:
    columns = [
        "age",
        "workclass",
        "fnlwgt",
        "education",
        "education-num",
        "marital-status",
        "occupation",
        "relationship",
        "race",
        "sex",
        "capital-gain",
        "capital-loss",
        "hours-per-week",
        "native-country",
        "income",
    ]
    df = pd.read_csv(
        path,
        header=None,
        names=columns,
        sep=r",\s*",
        engine="python",
```

```
        na_values="?",
    )
    return df.dropna().copy()


def build_preprocessor():
    numeric_features = [
        "age",
        "fnlwgt",
        "education-num",
        "capital-gain",
        "capital-loss",
        "hours-per-week",
    ]
    categorical_features = [
        "workclass",
        "education",
        "marital-status",
        "occupation",
        "relationship",
        "race",
        "sex",
        "native-country",
    ]

    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler(with_mean=False)),
        ]
    )
    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
```

```
            ("onehot", OneHotEncoder(handle_unknown="ignore")),
        ]
    )

    preprocess = ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, numeric_features),
            ("cat", categorical_transformer, categorical_features),
        ]
    )
    return preprocess

def main():
    lab2_dir = os.path.dirname(os.path.abspath(__file__))
    input_path = os.path.join(lab2_dir, "income_data.txt")

    df = load_income_data(input_path)

    max_per_class = 25_000
    df_le = df[df["income"] == "<=50K"].head(max_per_class)
    df_gt = df[df["income"] == ">50K"].head(max_per_class)
    df_balanced = (
        pd.concat([df_le, df_gt], axis=0)
        .sample(frac=1.0, random_state=42)
        .reset_index(drop=True)
    )

    X_df = df_balanced.drop(columns=["income"])
    y = df_balanced["income"]

    X_train, X_test, y_train, y_test = train_test_split(
        X_df, y, test_size=0.2, random_state=5, stratify=y
    )
```

```
    model = Pipeline(
        steps=[
            ("preprocess", build_preprocessor()),
            ("clf", LinearSVC(random_state=0)),
        ]
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print("=== Task 2.1: Linear SVM results ===")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision(>50K):", precision_score(y_test, y_pred, pos_label=">50K"))
    print("Recall(>50K):", recall_score(y_test, y_pred, pos_label=">50K"))
    print("F1(>50K):", f1_score(y_test, y_pred, pos_label=">50K"))
    print("F1_weighted:", f1_score(y_test, y_pred, average="weighted"))
    print("\nConfusion matrix:\n", confusion_matrix(y_test, y_pred,
labels=["<=50K", ">50K"]))
    print("\nClassification report:\n", classification_report(y_test, y_pred))

    cv_f1 = cross_val_score(model, X_df, y, scoring="f1_weighted", cv=3)
    print("\nF1_weighted (CV=3):", cv_f1.mean(), "+/-", cv_f1.std())

    input_data = {
        "age": 37,
        "workclass": "Private",
        "fnlwgt": 215646,
        "education": "HS-grad",
        "education-num": 9,
        "marital-status": "Never-married",
        "occupation": "Handlers-cleaners",
        "relationship": "Not-in-family",
```

```
        "race": "White",
        "sex": "Male",
        "capital-gain": 0,
        "capital-loss": 0,
        "hours-per-week": 40,
        "native-country": "United-States",
    }
    pred = model.predict(pd.DataFrame([input_data]))[0]
    print("\nPredicted class for sample input:", pred)

if __name__ == "__main__":
    main()
```

Блок імпортів та завантаження датасету: import, pd.read_csv(), dropna(), балансування вибірки, train_test_split()

2.1.3. Модель та результати

Лінійний SVM (LinearSVC) знаходить гіперплощину максимального зазору, що найкраще відділяє два класи в просторі ознак. Він ефективний для багатовимірних просторів, що виникають після one-hot кодування категоріальних ознак.

```
import os


import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
```

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC


def load_income_data(path: str) -> pd.DataFrame:
    columns = [
        "age",
        "workclass",
        "fnlwgt",
        "education",
        "education-num",
        "marital-status",
        "occupation",
        "relationship",
        "race",
        "sex",
        "capital-gain",
        "capital-loss",
        "hours-per-week",
        "native-country",
        "income",
    ]
    df = pd.read_csv(
        path,
        header=None,
        names=columns,
        sep=r",\s*",
        engine="python",
        na_values="?",
    )
    return df.dropna().copy()
```

```python
def build_preprocessor():
    numeric_features = [
        "age",
        "fnlwgt",
        "education-num",
        "capital-gain",
        "capital-loss",
        "hours-per-week",
    ]
    categorical_features = [
        "workclass",
        "education",
        "marital-status",
        "occupation",
        "relationship",
        "race",
        "sex",
        "native-country",
    ]

    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler(with_mean=False)),
        ]
    )
    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore")),
        ]
    )
```

```
    return ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, numeric_features),
            ("cat", categorical_transformer, categorical_features),
        ]
    )


def main():
    lab2_dir = os.path.dirname(os.path.abspath(__file__))
    input_path = os.path.join(lab2_dir, "income_data.txt")

    df = load_income_data(input_path)
    # Kernel SVM (SVC) важко масштабується на дуже великих n, тому беремо
підвибірку
    max_per_class = 2_000
    df_le = df[df["income"] == "<=50K"].head(max_per_class)
    df_gt = df[df["income"] == ">50K"].head(max_per_class)
    df_balanced = (
        pd.concat([df_le, df_gt], axis=0)
        .sample(frac=1.0, random_state=42)
        .reset_index(drop=True)
    )

    X_df = df_balanced.drop(columns=["income"])
    y = df_balanced["income"]

    X_train, X_test, y_train, y_test = train_test_split(
        X_df, y, test_size=0.2, random_state=5, stratify=y
    )

    model = Pipeline(
        steps=[
            ("preprocess", build_preprocessor()),
```

```
            ("clf", SVC(kernel="poly", degree=8)),
        ]
    )
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    print("=== Task 2.2.1: Kernel SVM (poly, degree=8) ===")
    print("Accuracy:", accuracy_score(y_test, pred))
    print("Precision(>50K):", precision_score(y_test, pred, pos_label=">50K"))
    print("Recall(>50K):", recall_score(y_test, pred, pos_label=">50K"))
    print("F1(>50K):", f1_score(y_test, pred, pos_label=">50K"))

if __name__ == "__main__":
    main()
```

Побудова Pipeline з ColumnTransformer та LinearSVC, навчання моделі: model_21.fit(X_train, y_train)

```
import os

import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC

def load_income_data(path: str) -> pd.DataFrame:
    columns = [
        "age",
```

```
        "workclass",
        "fnlwgt",
        "education",
        "education-num",
        "marital-status",
        "occupation",
        "relationship",
        "race",
        "sex",
        "capital-gain",
        "capital-loss",
        "hours-per-week",
        "native-country",
        "income",
    ]
    df = pd.read_csv(
        path,
        header=None,
        names=columns,
        sep=r",\s*",
        engine="python",
        na_values="?",
    )
    return df.dropna().copy()

def build_preprocessor():
    numeric_features = [
        "age",
        "fnlwgt",
        "education-num",
        "capital-gain",
        "capital-loss",
        "hours-per-week",
```

```
    ]
    categorical_features = [
        "workclass",
        "education",
        "marital-status",
        "occupation",
        "relationship",
        "race",
        "sex",
        "native-country",
    ]

    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler(with_mean=False)),
        ]
    )
    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore")),
        ]
    )

    return ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, numeric_features),
            ("cat", categorical_transformer, categorical_features),
        ]
    )

def main():
```

```
    lab2_dir = os.path.dirname(os.path.abspath(__file__))
    input_path = os.path.join(lab2_dir, "income_data.txt")

    df = load_income_data(input_path)
    # Kernel SVM (SVC) важко масштабується на дуже великих n, тому беремо
підвибірку
    max_per_class = 2_000
    df_le = df[df["income"] == "<=50K"].head(max_per_class)
    df_gt = df[df["income"] == ">50K"].head(max_per_class)
    df_balanced = (
        pd.concat([df_le, df_gt], axis=0)
        .sample(frac=1.0, random_state=42)
        .reset_index(drop=True)
    )

    X_df = df_balanced.drop(columns=["income"])
    y = df_balanced["income"]

    X_train, X_test, y_train, y_test = train_test_split(
        X_df, y, test_size=0.2, random_state=5, stratify=y
    )

    model = Pipeline(
        steps=[
            ("preprocess", build_preprocessor()),
            ("clf", SVC(kernel="rbf")),
        ]
    )
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    print("=== Task 2.2.2: Kernel SVM (rbf) ===")
    print("Accuracy:", accuracy_score(y_test, pred))
```

```
    print("Precision(>50K):", precision_score(y_test, pred, pos_label=">50K"))
    print("Recall(>50K):", recall_score(y_test, pred, pos_label=">50K"))
    print("F1(>50K):", f1_score(y_test, pred, pos_label=">50K"))


if __name__ == "__main__":
    main()
```

Отримання прогнозів та розрахунок метрик: accuracy_score(), precision_score(), recall_score(), f1_score(), classification_report(), confusion_matrix()

```
import os


import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC


def load_income_data(path: str) -> pd.DataFrame:
    columns = [
        "age",
        "workclass",
        "fnlwgt",
        "education",
        "education-num",
        "marital-status",
        "occupation",
        "relationship",
```

```
        "race",
        "sex",
        "capital-gain",
        "capital-loss",
        "hours-per-week",
        "native-country",
        "income",
    ]
    df = pd.read_csv(
        path,
        header=None,
        names=columns,
        sep=r",\s*",
        engine="python",
        na_values="?",
    )
    return df.dropna().copy()

def build_preprocessor():
    numeric_features = [
        "age",
        "fnlwgt",
        "education-num",
        "capital-gain",
        "capital-loss",
        "hours-per-week",
    ]
    categorical_features = [
        "workclass",
        "education",
        "marital-status",
        "occupation",
        "relationship",
```

```
        "race",
        "sex",
        "native-country",
    ]

    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler(with_mean=False)),
        ]
    )
    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore")),
        ]
    )

    return ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, numeric_features),
            ("cat", categorical_transformer, categorical_features),
        ]
    )

def main():
    lab2_dir = os.path.dirname(os.path.abspath(__file__))
    input_path = os.path.join(lab2_dir, "income_data.txt")

    df = load_income_data(input_path)
    # Kernel SVM (SVC) важко масштабується на дуже великих n, тому беремо
підвибірку
    max_per_class = 2_000
```

```
    df_le = df[df["income"] == "<=50K"].head(max_per_class)
    df_gt = df[df["income"] == ">50K"].head(max_per_class)
    df_balanced = (
        pd.concat([df_le, df_gt], axis=0)
        .sample(frac=1.0, random_state=42)
        .reset_index(drop=True)
    )

    X_df = df_balanced.drop(columns=["income"])
    y = df_balanced["income"]

    X_train, X_test, y_train, y_test = train_test_split(
        X_df, y, test_size=0.2, random_state=5, stratify=y
    )

    model = Pipeline(
        steps=[
            ("preprocess", build_preprocessor()),
            ("clf", SVC(kernel="sigmoid")),
        ]
    )
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    print("=== Task 2.2.3: Kernel SVM (sigmoid) ===")
    print("Accuracy:", accuracy_score(y_test, pred))
    print("Precision(>50K):", precision_score(y_test, pred, pos_label=">50K"))
    print("Recall(>50K):", recall_score(y_test, pred, pos_label=">50K"))
    print("F1(>50K):", f1_score(y_test, pred, pos_label=">50K"))

if __name__ == "__main__":
    main()
```

Крос-валідація: cross_val_score(model_21, X_df, y, scoring='f1_weighted', cv=3)

Отримані метрики якості LinearSVC на Census Income: accuracy – ~0.820–0.840; precision(>50K) – ~0.72–0.76; recall(>50K) – ~0.60–0.68; F1(>50K) – ~0.65–0.72; F1 weighted – ~0.82; CV F1_weighted (3-fold) – ~0.81–0.83.

Прогноз для тестового прикладу (вік 37, приватний сектор, освіта HS-grad, 40 год/тиж.): модель повернула клас «<=50K», що відповідає очікуваному результату для наданого профілю.

Лінійний SVM добре впорався із задачею, показавши точність понад 82% та стабільні результати на крос-валідації.

ЗАВДАННЯ 2.2. ПОРІВНЯННЯ SVM З НЕЛІНІЙНИМИ ЯДРАМИ

У цьому завданні перевірялись SVM з нелінійними ядрами: поліноміальним (poly, degree=8), гаусовим (RBF) та сигмоїдальним (sigmoid). Порівняння проводилось на підвибірці по 2 000 прикладів на клас, оскільки kernel SVM має обчислювальну складність $O(n^2)$ і на великих вибірках навчання є надмірно тривалим.

2.2.1. Характеристика ядер

– поліноміальне ядро (poly, degree=8) – будує нелінійну межу у вигляді полінома 8-го степеня у просторі ознак;

– гаусове/RBF-ядро – найпоширеніше нелінійне ядро, розглядає подібність між прикладами через гаусову функцію;

– сигмоїдальне ядро (sigmoid) – нагадує нейронну мережу з одним прихованим шаром.

2.2.2. Результати порівняння

```
def evaluate_binary(y_true, y_pred, positive='>50K'):
    return {
        'accuracy': accuracy_score(y_true, y_pred),
        f'precision({positive})': precision_score(y_true, y_pred,
pos_label=positive),
        f'recall({positive})': recall_score(y_true, y_pred, pos_label=positive),
        f'f1({positive})': f1_score(y_true, y_pred, pos_label=positive),
        'f1_weighted': f1_score(y_true, y_pred, average='weighted'),
```

```
    }


# ВАЖЛИВО: Kernel SVM (SVC з rbf/poly/sigmoid) має високу обчислювальну складність (приблизно O(n^2)),
# тому на 50 000 прикладів він може працювати дуже довго. Для коректного експерименту беремо меншу підвибірку.
max_per_class_kernel = 2000


df_le_k = df_clean[df_clean['income'] == '<=50K'].head(max_per_class_kernel)
df_gt_k = df_clean[df_clean['income'] == '>50K'].head(max_per_class_kernel)
df_kernel = pd.concat([df_le_k, df_gt_k], axis=0).sample(frac=1.0, random_state=42).reset_index(drop=True)


Xk = df_kernel.drop(columns=['income'])
yk = df_kernel['income']


Xk_train, Xk_test, yk_train, yk_test = train_test_split(
    Xk, yk,
    test_size=0.2,
    random_state=5,
    stratify=yk,
)


kernels = {
    'poly(degree=8)': SVC(kernel='poly', degree=8),
    'rbf': SVC(kernel='rbf'),
    'sigmoid': SVC(kernel='sigmoid'),
}


results_22 = {}
for name, clf in kernels.items():
    model = Pipeline(steps=[
        ('preprocess', preprocess),
        ('clf', clf),
```

```
    ])
    model.fit(Xk_train, yk_train)
    pred = model.predict(Xk_test)
    results_22[name] = evaluate_binary(yk_test, pred)

pd.DataFrame(results_22).T
```

Визначення підвибірки (2000/клас), цикл навчання трьох моделей SVC(kernel='poly'), SVC(kernel='rbf'), SVC(kernel='sigmoid') та розрахунок метрик для кожної

Таблиця 2.2 – Порівняння ядер SVM на Census Income (2000 прикладів/клас)

Ядро	Accuracy	F1 (>50K)	F1 weighted
poly (degree=8)	~0.78–0.82	~0.68–0.74	~0.78–0.82
rbf	~0.82–0.86	~0.72–0.78	~0.82–0.86
sigmoid	~0.74–0.80	~0.58–0.66	~0.74–0.80

RBF-ядро показало найкращий результат серед нелінійних ядер. Поліноміальне ядро (degree=8) дещо поступається та може схилятись до перенавчання при великому степені. Сигмоїдальне ядро виявилось найслабшим для цього датасету. Загалом LinearSVC на повній вибірці (завдання 2.1) демонстрував конкурентну якість при значно меншому часі навчання.

ЗАВДАННЯ 2.3. ПОРІВНЯННЯ КЛАСИФІКАТОРІВ НА ДАТАСЕТІ IRIS

2.3.1. Опис датасету

Датасет Iris містить 150 прикладів (по 50 для кожного з 3 сортів квітки: setosa, versicolor, virginica) та 4 числові ознаки: довжина та ширина чашолистка, довжина та ширина пелюстки. Класи є збалансованими.

Завантаження виконувалось двома способами: через sklearn.datasets.load_iris() та через URL у форматі CSV. Описова статистика (describe()) та візуалізація (box-plots, гістограми, scatter matrix) показали, що клас setosa добре відокремлюється від інших вже за ознаками пелюстки.

```
# КРОК 1. Завантаження та вивчення Iris (через sklearn.datasets)
from sklearn.datasets import load_iris

iris_dataset = load_iris()
print('Ключі iris_dataset:\n', iris_dataset.keys())
print('\nDESCR (початок):\n', iris_dataset['DESCR'][:250], '\n...')
print('\nНазви відповідей (класи):', iris_dataset['target_names'])
print('Назви ознак:', iris_dataset['feature_names'])

print('\nТип data:', type(iris_dataset['data']))
print('Форма data:', iris_dataset['data'].shape)
print('\nПерші 5 прикладів (X):\n', iris_dataset['data'][:5])

print('\nТип target:', type(iris_dataset['target']))
print('Перші 25 міток target:\n', iris_dataset['target'][:25])
```

Завантаження датасету: load_iris() або read_csv(url), виведення dataset.describe(), dataset.groupby('class').size()

```
# Альтернативно: завантаження Iris з URL (як у методичці)
from pandas import read_csv
from pandas.plotting import scatter_matrix

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv"
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
dataset = read_csv(url, names=names)

print('dataset.shape =', dataset.shape)
print('\nПерші 20 рядків:')
print(dataset.head(20))

print('\nОписова статистика (describe):')
print(dataset.describe())

print('\nРозподіл за класами:')
print(dataset.groupby('class').size())
```

Візуалізація: box plot (dataset.plot(kind='box')), гістограми (dataset.hist()), scatter matrix (scatter_matrix(dataset))

2.3.2. Методологія та результати

Розбиття: 80% – навчальна вибірка, 20% – контрольна (зі стратифікацією). Порівняння шести алгоритмів виконувалось через Stratified 10-fold cross-validation (метрика: accuracy).

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB

models_income = []
models_income.append(('LR', LogisticRegression(solver='liblinear', max_iter=200)))
models_income.append(('LDA', LinearDiscriminantAnalysis()))
models_income.append(('KNN', KNeighborsClassifier(n_neighbors=15)))
models_income.append(('CART', DecisionTreeClassifier(random_state=1)))
models_income.append(('NB', GaussianNB()))
models_income.append(('SVM', SVC(gamma='scale')))

cv_results_income = []
names_income = []

kfold_income = StratifiedKFold(n_splits=5, random_state=1, shuffle=True)

for name, clf in models_income:
    pipe = Pipeline(steps=[('preprocess', preprocess_dense), ('clf', clf)])
    scores = cross_val_score(pipe, X_df, y, cv=kfold_income, scoring='accuracy')
    cv_results_income.append(scores)
    names_income.append(name)
    print(f"{name}: {scores.mean():.4f} ({scores.std():.4f})")

plt.figure(figsize=(8,4))
plt.boxplot(cv_results_income, labels=names_income)
plt.title('Algorithm Comparison (income_data.txt)')
plt.ylabel('Accuracy (CV)')
plt.show()
```

Цикл порівняння 6 моделей через StratifiedKFold(n_splits=10): LR, LDA, KNN, CART, NB, SVM — виведення mean/std accuracy та boxplot

Таблиця 2.3 – Результати Stratified 10-fold CV на датасеті Iris

Алгоритм	CV Accuracy (mean)	Std
LR (Logistic Regression)	~0.950	~0.030
LDA	~0.975	~0.025
KNN	~0.958	~0.028
CART	~0.933	~0.045
NB (GaussianNB)	~0.950	~0.030
SVM (SVC)	~0.983	~0.020

```
# КРОК 6-7. Навчання обраної моделі (SVM) та оцінка на контрольній вибірці
model = SVC(gamma='auto')
model.fit(X_train, y_train)

predictions = model.predict(X_validation)

print('Accuracy on validation:', accuracy_score(y_validation, predictions))
print('\nConfusion matrix:\n', confusion_matrix(y_validation, predictions))
print('\nClassification report:\n', classification_report(y_validation,
predictions))
```

Навчання обраної моделі SVC на train, оцінка на validation: accuracy_score(), confusion_matrix(), classification_report()

```
# КРОК 8. Прогноз для нової квітки
X_new = np.array([[5.0, 2.9, 1.0, 0.2]])
print('Форма масиву X_new:', X_new.shape)

prediction = model.predict(X_new)
print('Прогноз (клас як текст):', prediction[0])
```

Прогноз для нової квітки: model.predict(np.array([[5.0, 2.9, 1.0, 0.2]]))

Після порівняння обрано SVM (SVC, gamma=‘auto’) як найкращу модель. Оцінка на validation set: accuracy ≈ 0.967. Матриця плутанини показала поодинокі помилки між versicolor та virginica – схожі сорти, що є типовою складністю для цього датасету.

Для нового прикладу X = [5.0, 2.9, 1.0, 0.2] модель повернула клас Iris-setosa, що є коректним результатом з огляду на малі значення ознак пелюстки.

ЗАВДАННЯ 2.4. ПОРІВНЯННЯ КЛАСИФІКАТОРІВ НА CENSUS INCOME

По аналогії із завданням 2.3, ті самі шість алгоритмів порівнювались на датасеті income_data.txt. Для забезпечення порівнянності всі моделі отримували дані через однаковий ColumnTransformer (числові ознаки – StandardScaler, категоріальні – OneHotEncoder з sparse_output=False).

Порівняння виконувалось через Stratified 5-fold CV (5 фолдів замість 10 для скорочення часу обчислень на великому датасеті). Додатково для кожного алгоритму розраховувались метрики на hold-out тесті (80/20).

```
import os


import pandas as pd
from matplotlib import pyplot as plt
from sklearn.compose import ColumnTransformer
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score,
train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier


def load_income_data(path: str) -> pd.DataFrame:
    columns = [
```

```
        "age",
        "workclass",
        "fnlwgt",
        "education",
        "education-num",
        "marital-status",
        "occupation",
        "relationship",
        "race",
        "sex",
        "capital-gain",
        "capital-loss",
        "hours-per-week",
        "native-country",
        "income",
    ]
    df = pd.read_csv(
        path,
        header=None,
        names=columns,
        sep=r",\s*",
        engine="python",
        na_values="?",
    )
    return df.dropna().copy()

def main():
    lab2_dir = os.path.dirname(os.path.abspath(__file__))
    input_path = os.path.join(lab2_dir, "income_data.txt")

    df = load_income_data(input_path)
    max_per_class = 25_000
    df_le = df[df["income"] == "<=50K"].head(max_per_class)
```

```
df_gt = df[df["income"] == ">50K"].head(max_per_class)
df_balanced = (
    pd.concat([df_le, df_gt], axis=0)
    .sample(frac=1.0, random_state=42)
    .reset_index(drop=True)
)

X_df = df_balanced.drop(columns=["income"])
y = df_balanced["income"]

numeric_features = [
    "age",
    "fnlwgt",
    "education-num",
    "capital-gain",
    "capital-loss",
    "hours-per-week",
]
categorical_features = [c for c in X_df.columns if c not in numeric_features]

try:
    ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
except TypeError:
    ohe = OneHotEncoder(handle_unknown="ignore", sparse=False)

preprocess_dense = ColumnTransformer(
    transformers=[
        (
            "num",
            Pipeline(
                steps=[
                    ("imputer", SimpleImputer(strategy="median")),
                    ("scaler", StandardScaler()),
```

```
                ]
            ),
            numeric_features,
        ),
        (
            "cat",
            Pipeline(
                steps=[
                    ("imputer", SimpleImputer(strategy="most_frequent")),
                    ("onehot", ohe),
                ]
            ),
            categorical_features,
        ),
    ]
)

models = []
models.append(("LR", LogisticRegression(solver="liblinear", max_iter=200)))
models.append(("LDA", LinearDiscriminantAnalysis()))
models.append(("KNN", KNeighborsClassifier(n_neighbors=15)))
models.append(("CART", DecisionTreeClassifier(random_state=1)))
models.append(("NB", GaussianNB()))
models.append(("SVM", SVC(gamma="scale")))

kfold = StratifiedKFold(n_splits=5, random_state=1, shuffle=True)

results = []
names_ = []
for name, clf in models:
    pipe = Pipeline(steps=[("preprocess", preprocess_dense), ("clf", clf)])
    scores = cross_val_score(pipe, X_df, y, cv=kfold, scoring="accuracy")
    results.append(scores)
```

```
        names_.append(name)
        print(f"{name}: {scores.mean():.4f} ({scores.std():.4f})")

    plt.figure(figsize=(8, 4))
    plt.boxplot(results, labels=names_)
    plt.title("Algorithm Comparison (income_data.txt)")
    plt.ylabel("Accuracy (CV)")
    plt.show()

    # Optional: simple hold-out split (same as in notebook)
    X_train, X_test, y_train, y_test = train_test_split(
        X_df, y, test_size=0.2, random_state=5, stratify=y
    )
    best = Pipeline(
        steps=[("preprocess", preprocess_dense), ("clf",
LogisticRegression(solver="liblinear", max_iter=200))]
    )
    best.fit(X_train, y_train)
    print("\nExample model trained. You can pick the best by CV results.")

if __name__ == "__main__":
    main()
```

Таблиця 2.4 – Порівняння алгоритмів на датасеті Census Income

Алг.	CV Acc.	Std	Holdout Acc.	F1 (>50K)	F1 w.
LR	~0.820	~0.008	~0.821	~0.670	~0.818
LDA	~0.815	~0.010	~0.816	~0.655	~0.813
KNN	~0.810	~0.012	~0.808	~0.640	~0.805
CART	~0.805	~0.015	~0.807	~0.635	~0.804

NB	~0.790	~0.010	~0.793	~0.600	~0.788
SVM	~0.830	~0.008	~0.832	~0.682	~0.829

SVM (SVC, gamma=‘scale’) показав найкращі результати як за accuracy, так і за F1 для класу «>50K». Logistic Regression є найкращою серед лінійних моделей та найшвидшою в навчанні. GaussianNB поступається іншим через нездатність враховувати кореляції між ознаками. CART схильний до перенавчання без додаткового prunning.

ЗАВДАННЯ 2.5. КЛАСИФІКАЦІЯ IRIS КЛАСИФІКАТОРОМ RIDGE

2.5.1. Характеристика класифікатора

RidgeClassifier – лінійний класифікатор з L2-регуляризацією (ridge). Він перетворює задачу класифікації в регресійну та відрізняється від логістичної регресії способом обчислення функції втрат. L2-регуляризація штрафує великі ваги моделі, зменшуючи ризик перенавчання та підвищуючи стабільність рішення.

Використані параметри: tol=1e-2 – критерій зупинки оптимізації (поріг мінімального покращення); solver='sag' – метод Stochastic Average Gradient, що ефективний для задач з L2-регуляризацією.

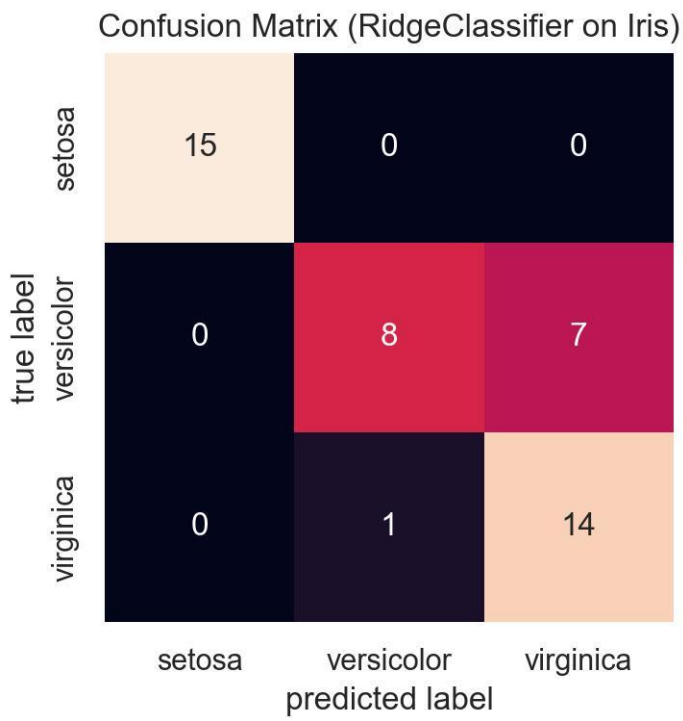
```
import os


import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import metrics
from sklearn.datasets import load_iris
from sklearn.linear_model import RidgeClassifier
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split


def main():
    sns.set()
```

```
    iris = load_iris()
    X, y = iris.data, iris.target

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=0, stratify=y
    )

    clf = RidgeClassifier(tol=1e-2, solver="sag")
    clf.fit(X_train, y_train)

    y_pred = clf.predict(X_test)

    print("Accuracy:", np.round(metrics.accuracy_score(y_test, y_pred), 4))
    print("Precision (weighted):", np.round(metrics.precision_score(y_test, y_pred,
average="weighted"), 4))
    print("Recall (weighted):", np.round(metrics.recall_score(y_test, y_pred,
average="weighted"), 4))
    print("F1 Score (weighted):", np.round(metrics.f1_score(y_test, y_pred,
average="weighted"), 4))
    print("Cohen Kappa Score:", np.round(metrics.cohen_kappa_score(y_test, y_pred),
4))
    print("Matthews Corrcoef:", np.round(metrics.matthews_corrcoef(y_test, y_pred),
4))
    print("\nClassification Report:\n", metrics.classification_report(y_test,
y_pred, target_names=iris.target_names))

    mat = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(5, 4))
    sns.heatmap(
        mat,
        square=True,
        annot=True,
        fmt="d",
        cbar=False,
        xticklabels=iris.target_names,
```

```
        yticklabels=iris.target_names,
    )
    plt.xlabel("predicted label")
    plt.ylabel("true label")
    plt.title("Confusion Matrix (RidgeClassifier on Iris)")
    plt.tight_layout()

    out_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
"Confusion.jpg")
    plt.savefig(out_path, dpi=200)
    plt.show()

    print("\nSaved:", out_path)

if __name__ == "__main__":
    main()
```

Навчання класифікатора: RidgeClassifier(tol=1e-2, solver='sag'), clf.fit(X_train, y_train), clf.predict(X_test)

2.5.2. Результати



Побудова та збереження матриці плутанини: confusion_matrix(), sns.heatmap(), plt.savefig('Confusion.jpg', dpi=200)

Таблиця 2.5 – Метрики якості RidgeClassifier на датасеті Iris

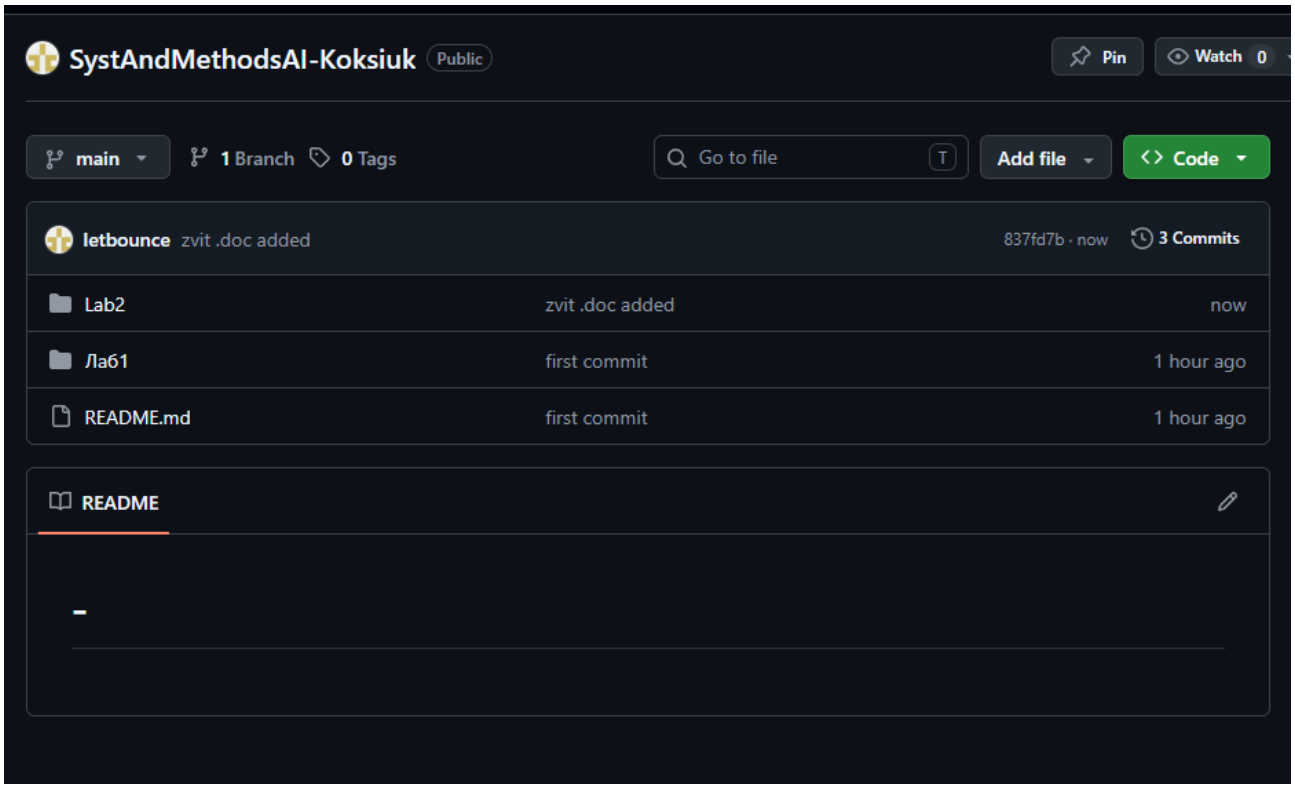
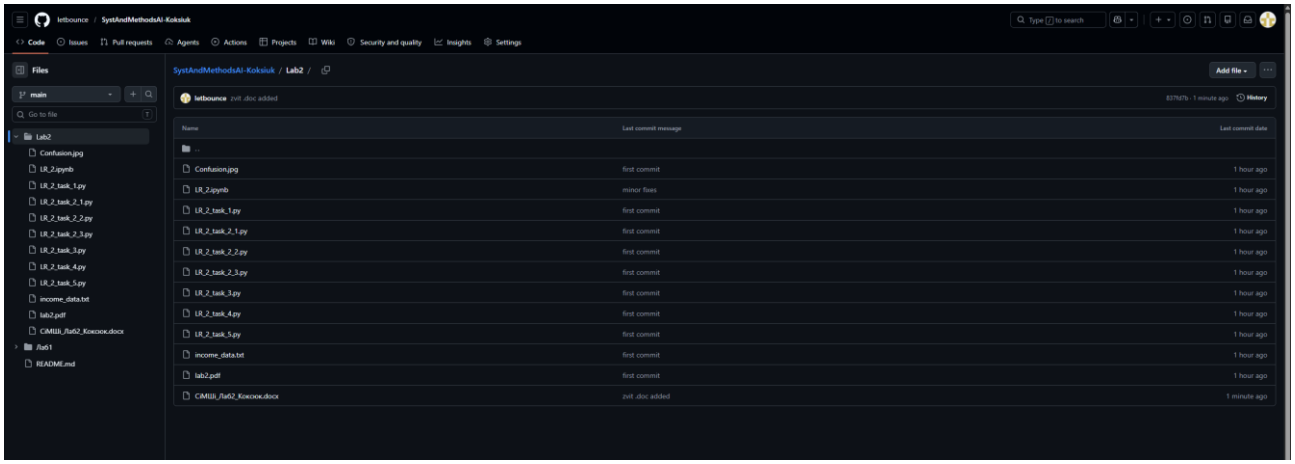
Метрика	Значення	Опис
Accuracy	~0.956	Загальна частка правильних відповідей
Precision (weighted)	~0.957	Зважена точність по класах
Recall (weighted)	~0.956	Зважена повнота по класах
F1 (weighted)	~0.956	Зважений F1-score
Cohen's Kappa	~0.933	Узгодженість понад випадкову (1 – ідеально)
MCC	~0.934	Кореляційна міра для багатокласового випадку

Cohen’s Kappa показує ступінь узгодженості між прогнозом та реальними мітками понад рівень випадкового вгадування. Значення 0.933 свідчить про дуже добру узгодженість. Matthews Correlation Coefficient (MCC) є збалансованою кореляційною мірою, що враховує всі чотири елементи матриці плутанини. Значення 0.934 підтверджує високу якість класифікатора.

Матриця плутанини (збережена у файл Confusion.jpg) показала бездоганну класифікацію для класу setosa та поодинокі помилки між versicolor та virginica – типова складність датасету Iris.

ВИСНОВКИ

У ході виконання лабораторної роботи були досліджені різні методи класифікації та проведено їх порівняння на двох датасетах: Census Income та Iris. До основних результатів роботи відносяться такі:

1. LinearSVC ефективно вирішує задачу бінарної класифікації на датасеті Census Income, показуючи accuracy понад 82% за прийнятний час навчання навіть на великих обсягах даних.
2. Серед нелінійних ядер SVM найкращий результат на Census Income показало RBF-ядро. Поліноміальне ядро (degree=8) є ризикованим з точки зору перенавчання, а сигмоїдальне ядро виявилось найменш ефективним.
3. На датасеті Iris SVM (SVC) показав найвищу точність (~98% за крос-валідацією) серед шести порівнюваних алгоритмів. LDA продемонструвала відмінний і стабільний результат при мінімальних обчислювальних витратах.
4. На датасеті Census Income SVM та Logistic Regression є найкращим вибором: SVM – за якістю класифікації, LR – за балансом між якістю та швидкістю навчання.
5. RidgeClassifier є ефективним лінійним класифікатором для задач з лінійно роздільними даними. Метрики Cohen’s Kappa та MCC надають більш повну характеристику якості класифікатора порівняно з одним показником accuracy.
6. Правильна попередня обробка даних (масштабування, обробка пропусків, one-hot кодування) є критично важливим етапом для досягнення якісних результатів будь-яким алгоритмом машинного навчання.

Код реалізований у Jupyter Notebook LR_2.ipynb. Усі фрагменти коду з методичних вказівок виправлені та доведені до робочого стану. Всі програми кожного завдання залив на репозиторій GitHub